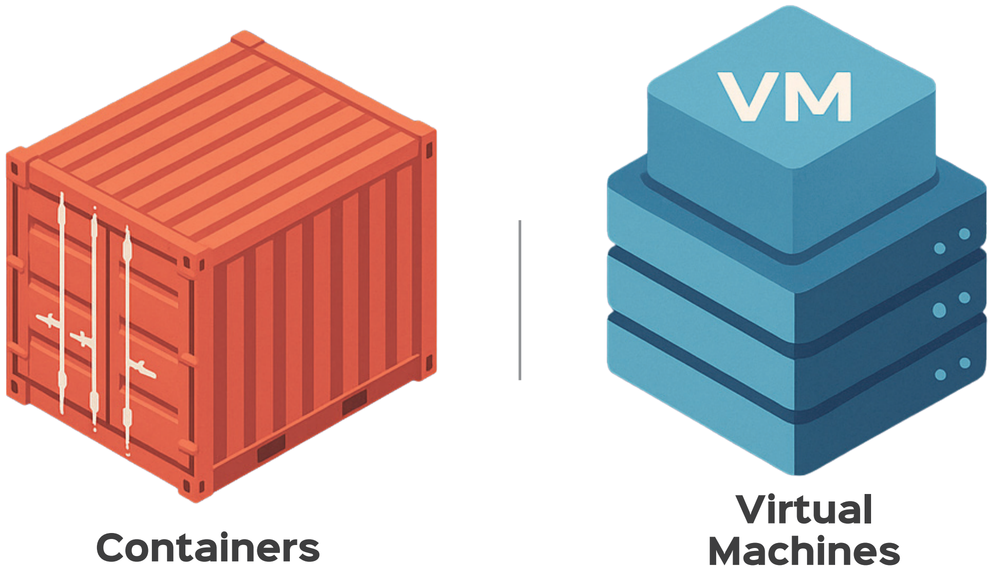
Containers and Virtual Machines: What's the Difference?

Here's an explanation of what containers and virtual machines are, how they work and compare, and when to use each technology in modern software deployment.



Many people ask, "Is a container just a lightweight version of a virtual machine (VM)?" While that may sound accurate at first, it is not correct. Containers and virtual machines may seem similar because both help in running applications in isolated environments, but they work in very different ways.

Teams often rely on these technologies to build, test, and deploy applications efficiently. Virtual machines and containers are powerful tools, but choosing the wrong one for a specific use case can lead to poor performance, higher costs, or security issues.

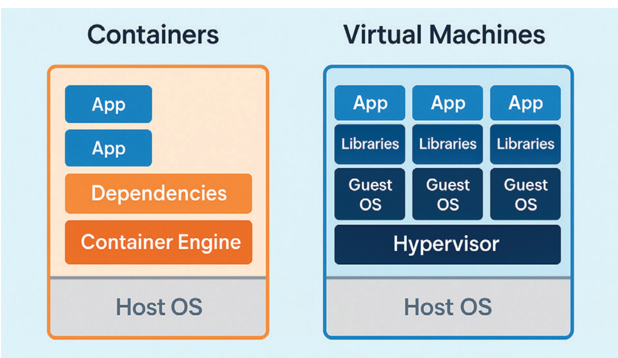


Figure 1: Containers vs VMs

That is why it is important to clearly understand how these two technologies differ. Knowing the differences helps developers, IT managers, and DevOps teams choose the right solution based on the goals of their project.

What is a virtual machine?

A virtual machine (VM) is a software-based environment that behaves like a physical computer. It allows you to run an operating system and applications just as you would on a regular desktop or server. VMs are created and managed using a tool called a hypervisor, which enables multiple virtual systems to run on a single physical machine.

How VMs work: A hypervisor sits between the hardware and the virtual machines. It allocates system resources such as CPU, memory, storage, and network access to each VM. Every VM includes a guest operating system, which is independent of the host system's OS. This allows VMs to run different operating systems on the same physical server.

For example, you can run a Linux VM on a Windows host machine. The hypervisor ensures each VM is isolated and protected from the others, providing a high level of separation. The benefits and limitations of VMs are listed in Table 1.